MOTHEO MOTSEMME AND KAI FYNN

TASK 1

1.1 movement System (state Pattern)

It is trying to be a state pattern, but instead it just uses a monolithic switch.

Map (Composite pattern)

It is trying to represent a part-whole Composite pattern, but instead it is a simple container (the Map object has aggregation relations with Region and Location).

Place features (Decorator pattern)

It is trying to be a decorator pattern, but instead it is a simple and messy inheritance hierarchy between Location (which is the parent) and (which are the children) features

Route Subsystem (Strategy pattern)

It's trying to be a State with RouteState and a giant if/else chain in getRoute(). It should be a Strategy Pattern. Routes should be interchangeable algorithms selected by the user. State is about behavior changing based on internal state transitions, while Strategy is about interchangeable algorithms selected externally. Routes are clearly Strategy because the user chooses which route calculation algorithm to use.

Biomes Subsystem (Abstract Factory)

It's trying to be an Abstract Factory with WorldBuilder and a giant switch statement. It should be an Abstract Factory Pattern, but the implementation is wrong. Abstract Factory should have an abstract factory interface with concrete factories per biome, not a single WorldBuilder with conditional logic.

The strategy pattern and the state pattern have been swapped, the intent of the traveller state pattern is to change the mode of transport based on the travellers internal state, on the other hand the strategy pattern's intent is vary the strategy of finding a route at runtime.

1.2 **state pattern:**

-GameManager : **Client**
-Traveller: **Context**
-Mode: **State**
-Foot, Bicycle, Car, Boat: **ConcreteState**

composite pattern:

-GameManager: **Client**
-Map : **Component**
-Location : **Leaf**
-Region: **Composite**

decorator pattern
-GameManager: **Client**
-MapElement : **Component**
-Location: **ConcreteComponent**
-MapElementDecorator : **Decorator**
-DragonDecorator, MountainsDecorator, ElfsDecorator, RainDecorator: **ConcreteDecorator**

Strategy Pattern Participants

- **Strategy:** RouteStrategy
- **Concrete Strategy:** FastestRoute, ScenicRoute, ShortestRoute
- **Context:** Trip

Abstract Factory Pattern Participants

- **Abstract Factory:** BiomeFactory
- **Concrete Factory:** DesertFactory, OceanFactory
- **Abstract Product:** NPC, Terrain, Obstacle
- **Concrete Product:** DesertNPC**,** OceanNPC**,** DesertTerrain**,** OceanTerrain**,** DesertObstacle**,** OceanObstacle

1.3

1. State and strategy have been swapped
Rule: (wrong pattern choice) The UML diagrams don't at all match the GoF diagrams,
Runtime consequences: the behaviour of the program will be unexpected since the wrong patterns are being used.

2. State pattern is missing concrete states, the UML diagrams are completely missing the various concrete state classes,
Rule: (wrong pattern choice) a large Switch statement is used to manage states, these large complicated constructs are not good coding practice.
Maintenance issues: Large switch statements are often difficult to maintain and can have logical errors if any dependencies are changed or tweaked.

3. Location class and Region class have aggregation arrows that contain asterisks in the middle of the relation line.
Rule: (wrong UML notation) asterisks should be at the end of the relation arrows
Maintenance Issues: any developers trying to use the Map class/edit anything will not know which class the asterisks apply to.

4. The Region class (which represents the Composite class) has incorrect ownership
Rule: (Broken ownership) the composite should be a Map (contain an "is-a" relationship) and should own maps too (contain a "has-a" relationship)
Runtime consequences: the tree data structure is lost and thus it won't behave as expected.

5. Wrong Pattern Choice in the “Abstract Factory”.
Rule: The WorldBuilder with a giant switch statement is not an Abstract Factory. The pattern requires an abstract factory interface with concrete factories per family.
Maintenance Issues: Adding a new biome requires modifying the WorldBuilder class, violating the Open/Closed Principle. The system cannot easily be extended without modifying the existing code.

6. No Abstract Classes or Virtual Destructors
Rule: The diagram shows no abstract classes and no virtual destructors declared.
Maintenance Issues: Declaring objects through base class pointers will cause memory leaks and undefined behavior. This violates the spec requirement that “destructors up a hierarchy are virtual.”

7. Giant Switch and conditional logic is used in WorldBuilder and MoveStrategy
Rule: WorldBuilder::make() uses a giant switch and MoveStrategy::doMove() uses a big switch to select behavior.
Maintenance Issues: This defeats the purpose of polymorphism. Adding new strategies or biomes requires modifying these switch statements, thus violating the Open/Closed Principle.

8. Excessive coupling. GameManager knows about everything.
Rule: GameManager knows about everything (mode, routeType, and has methods for everything: move(), plan(), build(), makeThing()).
Maintenance Issues: The GameManager is tightly coupled to all subsystems. Changes in any subsystem require changes to GameManager, making the system hard to maintain.

1.4

Tight coupling: every object becomes highly dependent on the GameManager, which results in code that is hard to maintain and breaks easily, making the central GameManager so tightly coupled it defeats the purpose of design patterns whose goal it is to ensure that all components are loosely coupled to each other.

Maintenance: the game manager can easily grow in size and complexity incredibly quickly making it hard to maintain when changes are made to the smaller classes which it handles/controls.

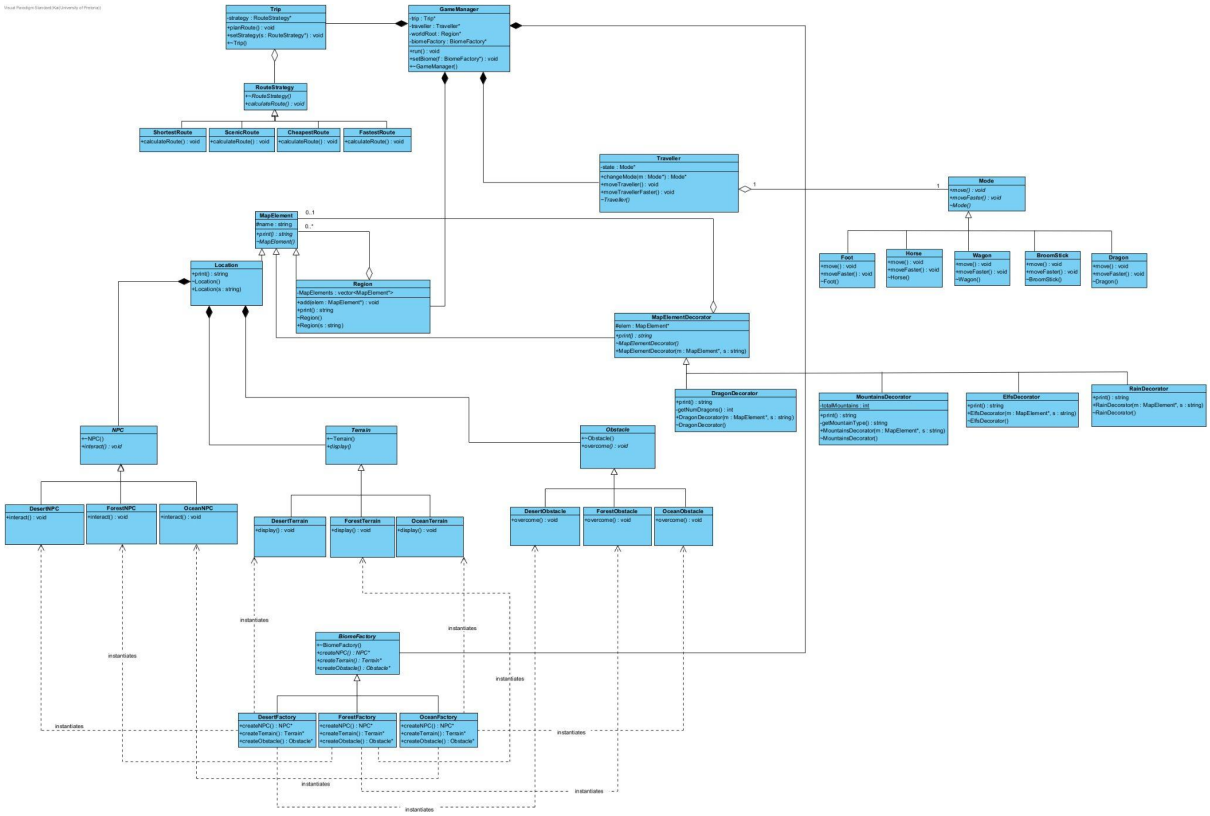
Ownership: The GameManager class has too much internal knowledge of all the other systems i.e Mode : int, RouteType : string, it also owns functions that should be internalised by the other subsystems. Instead the mode should belong to the state pattern, routeType should belong to the Strategy pattern.

1.5

The diagram is incredibly poorly made, there are lines drawn crossing over other class rectangles. Many of the same arrow heads point along object rectangles, this looks untidy and unprofessional, instead we should have a single arrow head that branches out to each related class.
The asterisks that relate Map, Location and Region are ambiguous (they are in the middle of the relation not at a particular end of the line).
Instead of diagonal straight lines, straight lines with 90 degree angles should be used to neatly indicate relationships.

Task 2

2.1



2.2

State pattern: The Traveller movement is handled by means of the state pattern. Each method of transport is modelled by a single concreteState class. The traveller contains a state member variable which is a pointer to a Mode object, which is the base class. Each mode specialization provides an implementation of the move() operation.

Composite pattern: The map is implemented by the composite design pattern. The base class is MapElement, it then has the Location and Region specializations which act as leaves and composites (respectively). Region (composite) manages its handles to other MapElements by means of a vector container. The tree data structure is modelled by the 0 to many relationship between the Region and MapElement classes.

Decorator pattern: the decorator pattern that I have created is a modified version of the classic Decorator pattern. The changes made have been made so as to enable the ability to have decorators applied to either Locations or Regions. The decorators have their own specialized operations and states.

Strategy Pattern: Trip (Context), RouteStrategy (Strategy), FastestRoute, ScenicRoute, ShortestRoute (Concrete Strategies).
Trip owns its RouteStrategy pointer and deletes it in the destructor. This ensures leak-free memory management and clear ownership semantics. When setStrategy() is called, the old strategy is deleted before assigning the new one, preventing memory leaks allowing strategies to switch at runtime.
The user should be able to select different route algorithms at runtime. These are interchangeable algorithms that achieve the same goal but use different criteria. The Strategy pattern is perfect because it allows algorithms to vary independently from clients that use them.

Abstract Factory Pattern: BiomeFactory (Abstract Factory), DesertFactory, OceanFactory (Concrete Factories), NPC, Terrain, Obstacle (Abstract Products), and their concrete implementations.
Each concrete factory creates and returns pointers to product objects. The caller (GameManager) owns these objects and is responsible for deleting them.
Biomes are families of related products (NPC, Terrain, Obstacle) that must be consistent within a biome. Abstract Factory ensures that products from the same family are used together, preventing the mixing of Desert NPCs with Ocean Terrain. The pattern provides an abstraction for creating these families, making the system extensible to new biomes without modifying existing code.

2.3.

Added a new Route Strategy, CheapestRoute. This new concrete strategy calculates the cheapest route. The new class inherits from RouteStrategy and implements calculateRoute(). No existing class needs to change because they adhere to the same interface. The user can select this strategy through the existing Trip::setStrategy() operation.

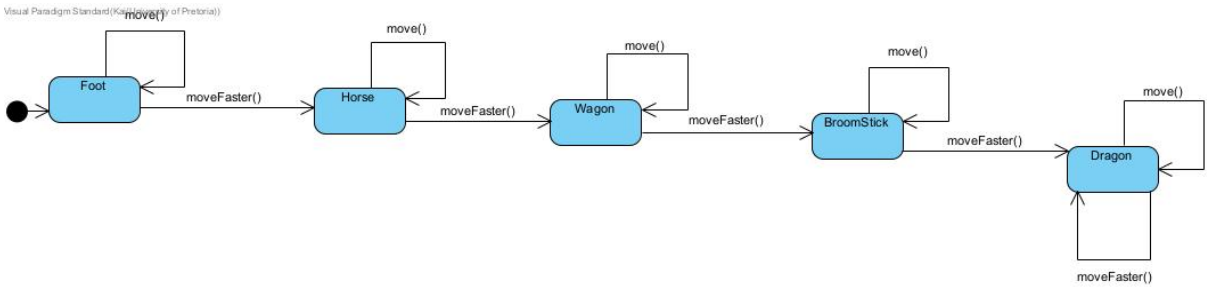
Added a new Route Strategy, ScenicRoute. This new concrete strategy calculates a route optimized for scenic views, passing through beautiful locations, natural landmarks, etc. This new class inherits from RouteStrategy and implements calculateRoute(). No existing classes

need to change because the Strategy pattern's interface remains stable. The system is extended by simply adding a new class that conforms to the existing abstract interface.

Added a new Biome, ForestFactory. This new biome factory produces forest-themed products: ForestNPC, ForestTerrain, and ForestObstacle. The new factory inherits from BiomeFactory and implements the three creation methods. No existing factories, products, or client code needs to change because they all depend on the abstract factories. The GameManager can simply be given a ForestFactory instance without any modifications.

Task 3:

3.3

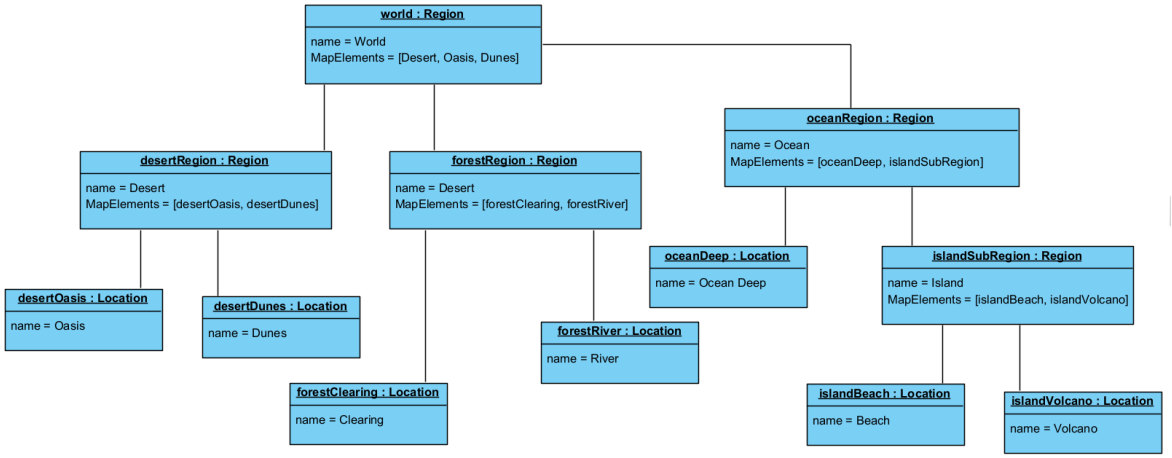


Task 4:

4.3. The Strategy and State patterns have similar class structures, however they serve different purposes. Routes are implemented using the Strategy pattern because route calculation represents a family of interchangeable algorithms(fastest,scenic,shortest) that the user selects based on their preferences. The Trip context delegates whatever strategy it's been given, and the strategy itself has no knowledge of other strategies or the ability to transition between them.
The State pattern allows an object to alter its behavior when its internal state changes. Movement is State because the Traveller's behavior changes based on its current travel mode, and each Concrete State (Foot, Horse, Wagon, BroomStick, Dragon) knows about subsequent states and triggers transitions through `moveFaster()`. The state itself controls its own lifecycle based on internal rules.
The two are not interchangeable because swapping them violates their intents. Using Strategy for movement would require the user to manually select every travel mode, breaking the upgrade chain and shifting the responsibility to the client. Using State for routes would force routes to transition internally, preventing users from explicitly choosing their preferred route type.

Task 5:

5.3



Task 6:

6.2